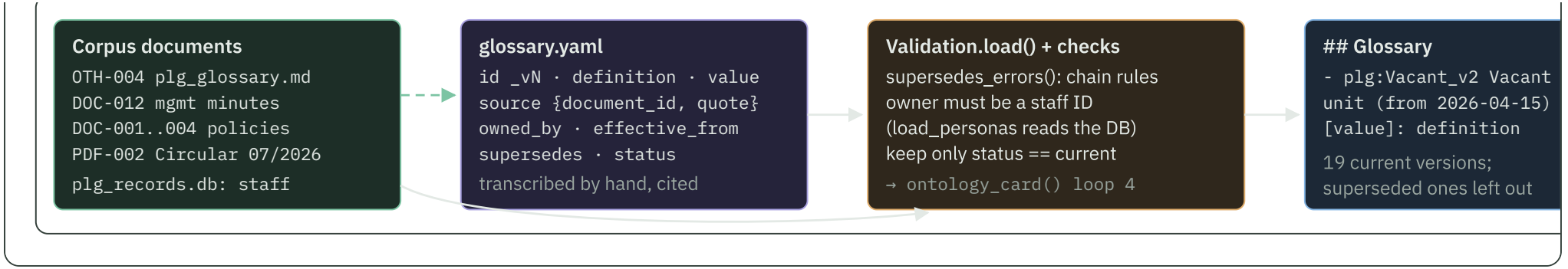
How ontology_card.md is made

The card is generated, but not from `poc-data` directly. It is printed by `scripts/build_ontology.py` from three YAML sources in `ontology/`. Claude Code wrote those YAML files once, from the solution design and `poc-data`, and tests keep them in line with poc-data. The build itself is plain code with no LLM call.





Walkthrough

1. `poc-data/` is the fictional corpus, made by `poc-data/_generator` (seed 42). Its ground truth lives in `_master/*.json` and the SQLite `plg_records.db`.
2. All of `ontology/` was written by Claude Code in one commit (`d926c09`, "Ontology v1 and business glossary seed (#4)", co-authored by Claude, 2026-10-13). It followed the design in `docs/solution.MD` §2.7 and the spec, and read poc-data while doing it: the PLG Records tables became node classes and their foreign keys became relations; the taxonomy, checklists and code lists became the vocabularies; the corpus docs were quoted into the glossary. Each vocab file still says `"# Seeded from poc-data ground truth; synonyms (alt_labels) curated by hand"`, but "by hand" here means in that session, not by a script. No generator for the YAML exists in the repo.
3. From then on `ontology/` is the source you edit (or ask Claude to edit): `plg.yaml` (LinkML classes and relations), `vocab/*.yaml` and `glossary.yaml`.
4. `make ontology` runs `build()` in `scripts/build_ontology.py`. It loads `plg.yaml`, generates Pydantic models, validates every vocab concept and glossary term against them, checks references and `supersedes` chains, and checks each glossary owner against the `staff` table. If anything fails it writes nothing.
5. If all checks pass, `ontology_card()` prints the four sections and writes `src/plg/ontology/generated/ontology_card.md`, next to `models.py`, `allow_list.json`, `vocab_index.json` and the other artefacts.

and the other artefacts.

6. `tests/test_ontology_artefacts.py` then checks the committed output against poc-data (taxonomy, checklists, trades, unit statuses and types, letter types, glossary owners and cited doc IDs). CI reruns the generator and fails if the committed files differ.

Where each section comes from

SECTION	KIND	SOURCE	CODE
Node labels	`AI-written once`	`plg.yaml` classes with `annotations: {node: true}`, plus `description`. Modelled on solution.MD §2.7 and the 17 tables in `poc-data/database/schema.sql`	`Ontology.__init__` (`self.nodes`), loop 1 of `ontology_card()`
Relations	`AI-written once`	`plg.yaml` slots with `edge`, `extracted`, `range`, `required`, `multivalued`. Mostly the FK columns of those tables, plus document edges from the design	`Ontology.edges()`, loop 2
Vocabularies	`seeded from data` + `hand synonyms`	`ontology/vocab/*.yaml`, seeded from `_master/taxonomy_and_checklists.json` and `plg_records.db` (`trades`, `units.status`, `units.unit_type`, `letters.letter_type`)	`Validation.load()`, `check_references()`, loop 3
Glossary	`AI-transcribed`, cited	`ontology/glossary.yaml`, quoting corpus docs by `_manifest.csv` ID (mostly OTH-004, DOC-001 to DOC-004, DOC-012, PDF-002)	`Validation.load()`, `supersedes_errors()`, owner check via `load_personas()`, loop 4 (current only)

The header line comes from the `HEADER` constant and the version from `plg.yaml`'s `version: "1.0.0"`. The section headings are hard-coded in `ontology_card()`.

Regenerate it

```
cd ~/Documents/DevProjects/agent-hdb-v2
uv sync                # once: dev deps include LinkML
make ontology          # = uv run python scripts/build_ontology.py
uv run pytest tests/test_ontology_artefacts.py
```

Then commit `src/plg/ontology/generated/`. Editing the card by hand is pointless, because the next build overwrites it and CI flags the drift.

Things that look off

- The card is in **agent-hdb-v2**. The `agent-hdb` folder has no ontology at all.
- `docs/solution.MD` (§2.7 and D12/D14) and `docs/spec.md` say 8 defect categories. The taxonomy in poc-data, the vocab file and the card all have 11 (STR to OTH). The data and card agree; the docs are stale.
- Only 7 of the 25 node labels have a description, so the card gives the agent little for Town, Block, Inspection, Task and the rest.
- Relations name `Document` and `Entity`, which are abstract and not in the Node labels list. The edge is declared on the parent class, so readers have to know Policy, Sop and Circular are Documents.
- `SUPERSEDES` is used for two different edges: Document to Document (extracted, marked *) and GlossaryTerm to GlossaryTerm (not extracted).
- Some vocabularies are not checked against poc-data by any test. `document_types` is curated (the manifest's `type` column holds file formats like pdf and xlsx, not these). `teams` and `vendors` are checked only through the taxonomy test. Glossary definitions are not compared with the quoted source text; tests check only that the cited doc IDs and owners exist.
- `unit_statuses` is titled "snapshot as at 2026-09-30". If poc-data is regenerated, the vocab needs reseeding by hand, and the tests are what will tell you.